



issue-opfs

Can I get an analysis and opinion of this issue?

This is a very well-structured and timely migration plan. Transitioning from the legacy `sql.js` in-memory/IndexedDB blob export pattern to an Origin Private File System (OPFS) backed architecture is the correct evolutionary step for a modern browser-based database. It addresses the fundamental flaw of the old model: the heavy performance and memory overhead of serializing and deserializing the entire database on every load and save.

MD + 1

Here is an analysis of the proposal, highlighting its strengths, potential architectural traps, and opinions on your key design questions.

Strengths of the Proposal

- **Scoped Migration Strategy:** Relying on the existing Supabase sync architecture to hydrate the new OPFS database—rather than writing a complex IndexedDB-to-OPFS local data migrator—is a highly pragmatic decision. It treats the local database as ephemeral for the sake of the migration, which vastly reduces risk.
- **Abstraction via oosync:** Building an OPFS-backed runtime path directly into the `oosync` library ensures the persistence mode remains an implementation detail, protecting the rest of the application's call sites.
- **Clear Testing Boundaries:** The acceptance criteria distinctly separate unit testing the OPFS initialization from testing the offline-first workflows and multi-tab coordination.

MD + 1

MD

MD

Architectural Challenges & "Gotchas"

There are a few sharp edges when implementing `@sqlite.org/sqlite-wasm` with OPFS, particularly within a Vite, SolidJS, and TypeScript ecosystem.

MD

1. The Web Worker and Drizzle Concurrency

OPFS access is synchronous, meaning `@sqlite.org/sqlite-wasm` highly recommends (and for the optimal VFS, requires) running the database inside a Web Worker.

Because Drizzle ORM call sites currently sit in the main thread, shifting SQLite to a worker

means Drizzle needs an asynchronous driver to communicate via `postMessage`. You may need to implement a custom Drizzle proxy/worker driver to bridge the main SolidJS UI thread and the SQLite worker thread.

MD + 1

2. COOP/COEP Headers

To achieve maximum performance, SQLite WASM utilizes `SharedArrayBuffer`, which mandates strict Cross-Origin Isolation headers (`Cross-Origin-Embedder-Policy: require-corp` and `Cross-Origin-Opener-Policy: same-origin`).

- **The Trap:** Enabling these headers can instantly break third-party integrations, particularly OAuth redirects or external media fetching. You will need to carefully review how these headers impact Supabase authentication flows.

3. The Multi-Tab Locking Problem

OPFS locks files exclusively. If Tab A opens `tunetrees.sqlite` in OPFS, Tab B will throw an error if it tries to initialize its own SQLite instance against the same file.

MD

Opinions on Key Design Questions

- **Dedicated, Shared, or Main Thread?**

It must be a Worker to utilize the synchronous OPFS API. A `SharedWorker` is often the cleanest solution for the multi-tab problem, acting as a local server that all tabs connect to. However, `SharedWorkers` have spotty support on mobile browsers (particularly Safari on iOS). If mobile usage is a priority, a `Dedicated Worker` per tab with leader-election (via `BroadcastChannel`) is a safer, albeit more complex, route.

- **Per-User DBs vs. Logical Separation?**

Per-user OPFS database files are highly recommended. It guarantees that clearing an OPFS file completely isolates the destruction of one user's data without risking another's, cleanly solving the local reset requirement.

MD + 1

- **Fallback Behavior?**

If OPFS is unavailable (e.g., in incognito modes or older browsers), dropping back to the existing in-memory + `IndexedDB` blob export is a perfectly valid fallback. You could retain the old persistence code strictly as the fallback adapter in `oosync` .

MD + 1

- **Playwright / E2E Resets:**

Playwright test contexts are entirely isolated. OPFS state is stored in the browser profile's user data directory. Launching a fresh browser context per E2E test will naturally yield an empty OPFS directory, solving deterministic resets without needing manual cleanup scripts.

MD

Given the strict file-locking nature of OPFS, are you currently leaning toward attempting a `SharedWorker` architecture for the multi-tab support, or implementing a leader-election

pattern where one active tab holds the lock?

By "multi-tab support" I assume you mean the browser tab level. I don't know SharedWorker vs leader-election, that's down the line.



Yes, by multi-tab support, that refers directly to a user having TuneTrees open in multiple browser tabs at the same time. The file-locking mechanism of OPFS forces you to handle how those tabs interact with the single database file, but that is definitely an implementation detail for later.

To answer your core question: moving to OPFS provides massive gains on **both** startup and transaction performance, but the fundamental advantage is shifting from an "ephemeral document" model to a true "database" model.

Here is a breakdown of what you actually gain.

Performance on Startup

With your current architecture (in-memory -> export bytes -> IndexedDB), your startup time is directly tied to the total size of your database.

- **Current State:** When the app loads, it has to fetch the entire blob from IndexedDB, allocate memory, and deserialize the entire byte array back into SQLite before the user can do anything. As the database grows, this initialization gets progressively slower.
- **The OPFS Gain:** OPFS allows SQLite to act like a normal, disk-backed database. On startup, SQLite merely opens a file handle. Initialization becomes a near-instant, constant-time operation regardless of whether the database is 1 MB or 500 MB.

Performance on Transactions

This is where the difference is most stark.

- **Current State:** If a user checks off a single practice session or updates a minor setting, you have to serialize the *entire* database to a byte array and overwrite the old IndexedDB blob. Writing a 50 MB blob to IndexedDB just to save a 10-byte string change is incredibly inefficient and locks the main thread (or worker) heavily.
- **The OPFS Gain:** OPFS utilizes a Virtual File System (VFS) that allows SQLite to do incremental, page-level writes. If you update one row, SQLite only writes a few kilobytes

directly to the OPFS file. You no longer have to debounce or batch your saves; you can just execute a SQL `UPDATE` and trust it is efficiently on disk in milliseconds.

Architectural Advantages Beyond Speed

Performance aside, adopting OPFS removes several fragile ceilings from the application.

- **Memory Overhead (The Mobile Killer):** Your current approach requires the entire database to live in RAM, and during the export process, that memory footprint temporarily doubles while the byte array is built. On a desktop browser, this is fine. On a mobile browser, aggressive memory management will quickly kill the tab if the memory spikes. OPFS pages data in and out of memory as needed, drastically lowering the application's resting RAM footprint.
- **Crash Durability:** With the in-memory/export model, if the browser crashes, the tab is aggressively suspended, or the battery dies before an IndexedDB export finishes, any data written since the last export is lost. OPFS commits transactions directly to the local disk. If a SQL write succeeds, it is durably saved, mimicking the reliability of a traditional server-side database.

Are you currently debouncing your IndexedDB exports to avoid locking the UI, or is the TuneTrees database still small enough that you are exporting on every single transaction?

I think some debouncing is going on. And, yes, there are some performance issue, though I haven't been sure if they're at the local DB level or the sync with supabase level. Possibly both I guess.

It makes complete sense that it is difficult to isolate the source of the performance hit. When an architecture combines local blob serialization with a network synchronization layer, a bottleneck in either phase can manifest to the user as general "app sluggishness."

When the local database and the network sync process are coupled too tightly, a lag spike can blur the lines between local UI thread locking and network I/O latency.

Here is a breakdown of how to mentally (and technically) decouple the two to figure out exactly what is slowing down.

1. Identifying Local DB Bottlenecks (The IndexedDB Export)

The defining characteristic of the SQLite-to-IndexedDB export process is that it is computationally heavy. Serialization involves reading the entire database from memory, converting it to a byte array, and writing that massive blob to disk.

- **The Symptom:** Hard UI freezing or stuttering. If you click a button in the UI to update a practice session, and the entire interface locks up for a fraction of a second (animations drop frames, buttons stay in a pressed state, scrolling hangs), the local export is the culprit.
- **The Test:** Temporarily disable the `oosync` push to Supabase entirely. Keep the app completely offline. If you still feel the lag or see the UI stutter when writing through Drizzle ORM, you are feeling the weight of the IndexedDB blob export.

2. Identifying Sync Bottlenecks (Supabase & Network I/O)

The defining characteristic of network sync is latency, not computation. Pushing your outbox records to Supabase over REST or WebSockets involves waiting for round-trip network times and database transaction times on the server.

- **The Symptom:** The UI remains perfectly fluid and responsive, but the "state" of the app feels delayed. For instance, you check off a tune, the UI immediately updates locally (thanks to SolidJS reactivity), but the sync indicator spins for a few seconds. If you have the app open on your 16-inch MacBook and another device, the delay before the data appears on the second screen is the sync overhead.
- **The Test:** Comment out the local IndexedDB export function so the database strictly lives in memory for a session. If the UI suddenly feels buttery smooth but the data takes a few seconds to officially settle with the server, your bottleneck is purely network latency and Supabase transaction times.

Why OPFS Clarifies Everything

One of the secondary benefits of migrating to OPFS is that it forces a clean separation of these concerns.

By utilizing OPFS, the local write becomes an instant, fractional-millisecond page update. Once that local bottleneck is permanently removed, any remaining performance issues are immediately isolated to the `oosync` and Supabase network layer. It turns a muddy performance problem into a highly observable one.

When you experience these performance issues, are you seeing the UI physically stutter and drop frames when you interact with it, or does the application remain responsive while the data just takes a noticeable moment to settle?

Well, I would say the actual problem isn't app sluggishness as such. It's time for sync to occur. First sync, or updates. From what I can tell, the indexed DB is 6.92 MB.

That makes perfect sense. At just under 7 MB, serializing that data to IndexedDB is still a relatively lightweight task for the CPU, especially when running on robust hardware like a 16-inch MacBook Pro. The CPU can crunch that byte array fast enough that the SolidJS UI doesn't physically choke.

Here is the candid reality: **migrating to OPFS will not solve this specific problem.**

OPFS optimizes the local disk write. It does absolutely nothing for network latency, payload serialization, or Supabase transaction times. If the friction is purely in the `oosync` pipeline—getting data up to the server or pulling the initial state down—your bottleneck is architectural at the network layer.

Here is why a ~7 MB database can feel slow to sync, and where you likely need to look within the `oosync` implementation:

1. The "First Sync" Latency

If an initial sync takes a noticeable amount of time, it is rarely the raw size of the data (7 MB over modern broadband takes a fraction of a second). The delay is usually the **request volume**.

If the application is querying Supabase and pulling down data table-by-table via standard REST endpoints to reconstruct the local SQLite state, the HTTP network overhead and Postgres query planning times stack up massively.

2. The Outbox "Updates" Problem

Similarly, when the local outbox pushes updates, how are they handled? If you update a practice log and edit a tune's metadata, does `oosync` fire separate `PATCH` requests to Supabase for each change? If so, you are paying the network round-trip penalty multiple times.

3. The JSON Serialization Tax

Your SQLite database might be 6.92 MB locally, but when that data is queried, converted to JSON, sent over the wire, and parsed back into SQLite `INSERT` statements, the processing overhead on both the client and the Supabase Postgres instance is significantly heavier than just moving a binary file.

The Path Forward

OPFS is still the correct long-term move for data durability and preventing future memory limits as TuneTrees grows. However, to solve the sync latency, the focus needs to shift to **Bulk Operations**.

For Supabase, the standard optimization is utilizing Postgres RPCs (Remote Procedure Calls). Instead of the client iterating through requests, `oosync` bundles all outbox changes into a single large JSON payload. It sends that one payload to a custom RPC endpoint, allowing the Postgres server to do the heavy lifting of bulk upserting the data in a single, lightning-fast transaction.

When pushing pending changes up to Supabase right now, are you utilizing the standard Supabase client methods (like chaining `.update()` on specific tables), or are you already sending batched payloads?

> If the application is querying Supabase and pulling down data table-by-table via standard REST endpoints to reconstruct the local SQLite state, the HTTP network overhead and Postgres query planning times stack up

That is a highly capable, modern sync architecture. Leveraging Cloudflare Hyperdrive to pool connections for edge workers is exactly the right move to mitigate the connection overhead of serverless functions.

Looking closely at this end-to-end flow, the "sluggishness" during initial syncs and updates is not a problem with the local SQLite footprint. The bottleneck is almost certainly rooted in **Edge-to-Database network latency accumulation**.

Here is an analysis of exactly where the time is being spent in this pipeline.

1. The PUSH Bottleneck: Sequential Round-Trips

During the PUSH phase, applying changes inside a single Drizzle transaction guarantees atomicity, but it does not guarantee speed if the operations are sequential.

If the worker loops through the `SyncChange` array and executes an `UPSERT` or `DELETE` for each item individually:

1. The worker sends the query to Supabase.
2. Postgres executes it.
3. Postgres sends the success signal back to the worker.

4. The loop advances to the next item.

Even with Hyperdrive keeping the connection warm, every single query incurs the physical network latency between the Cloudflare Edge location and the Supabase datacenter. If the latency is 30ms, and you push 100 changes, you instantly accumulate 3 seconds of strict waiting time, entirely separate from database execution time.

The Fix: Batching. Drizzle supports batch inserts. Group the `SyncChange` objects by table and execute a single `UPSERT` per table passing an array of values.

2. The Initial PULL Bottleneck: The Pagination Tax

For a ~7MB database, an initial sync can feel agonizingly slow due to the pagination strategy.

Paginating 500 rows per page until all tables are exhausted requires the browser to make multiple, consecutive HTTP `POST /api/sync` requests. Every single request incurs:

- Browser to Cloudflare network transit.
- Worker initialization and JWT verification.
- Hyperdrive connection acquisition.
- The Drizzle transaction setup.
- The query execution and JSON serialization.

If a 7MB database translates to 15,000 rows across various tables, that is 30 separate HTTP requests just to pull the initial state. The overhead of the HTTP protocol and transaction setup dwarfs the actual data transfer time.

The Fix: For initial syncs, consider vastly increasing the page size (e.g., 5,000 rows), or shifting to a streaming response where the worker queries all tables and streams the JSON chunks back in a single HTTP request.

3. The Prepared Statement Penalty

The `postgres-js` configuration `prepare: false` is a common requirement when using transaction-pooling (like PgBouncer), but it comes with a performance penalty.

By disabling prepared statements, Postgres must parse, analyze, and plan the SQL query for every single `UPSERT` and `SELECT` operation, every time it runs. For repetitive sync operations, the database is spending CPU cycles constantly re-planning identical queries rather than executing them.

The Fix: Verify if Cloudflare Hyperdrive's current iteration supports prepared statements with your specific Supabase configuration. If it does, enabling it will significantly reduce the CPU load and execution time on the Supabase Postgres instance.

4. The RPC Advantage

Your architecture already uses Postgres RPCs for complex tables to bypass the Drizzle query builder. This is highly efficient because it executes raw SQL closer to the metal.

If optimizing Drizzle batching proves complex for the PUSH phase, extending that RPC pattern so the Cloudflare Worker hands the entire JSON array of `SyncChange` objects directly to a single Postgres function would eliminate the Edge-to-Database latency entirely. The worker makes one network call, and Postgres processes the entire loop internally.

Yeah, I was thinking about the sync check being done on the supabase server side also. I think I might need to go to the \$100 Codex max plan for the month to pull something like that off. Seems like a significant re-architecture of oosync 

Your roadmap is practically textbook for safe, incremental systems engineering. Attempting to change the local storage engine and the network synchronization protocol simultaneously is a guaranteed way to create a debugging nightmare.

Here is exactly why this sequence makes total sense:

Phase 1: Stabilize the Engine Swap

Leaving the old IndexedDB persistence pattern in place while swapping the core engine to modern SQLite WASM is the perfect isolation strategy. If a local database resets or a read fails tomorrow, you know with absolute certainty it is an artifact of the WASM integration, not a weird file-locking issue from a brand new OPFS implementation. Letting this bake and settle ensures a rock-solid foundation.

Phase 2: The `oosync` Refactor (The Watermelon)

Tackling the network bottleneck next addresses the actual user-facing pain point. Shifting to bulk syncs and raw Postgres RPCs is a substantial architectural lift, but it is exactly the kind of dense, boilerplate-heavy transition where upgrading to a premium AI plan pays for itself.

A \$100 monthly tier for a tool like Codex or the Claude Code Max plan provides massive token allowances and extended session lengths. Pointing an unrestricted agent at the codebase, equipped with your Model Context Protocol servers running locally on the 16-inch MacBook Pro, will absolutely accelerate the tedious work of rewriting those sequential Drizzle loops into batched payloads.

Bankless

Phase 3: The OPFS Migration

Once the network sync is blindingly fast and fully stabilized, you can finally rip out the IndexedDB blob export. Moving to OPFS becomes the final layer of polish, permanently solving local memory constraints and crash durability without the risk of breaking your newly optimized sync pipeline.

When you do begin the `oosync` batching refactor, are you planning to handle offline conflict resolution entirely within those new Postgres RPCs, or will the Cloudflare Worker still manage the "last-writer-wins" logic before it hands the data to the database?

Ok, can you write an issue description that includes problem statement + proposal? I know you don't have visibility into my app, but I've shown you the architecture such that you have a pretty good idea. I just want to make sure I capture your 

Here is a comprehensive issue draft that captures the performance analysis and the structural proposal. You can drop this straight into your tracking system or markdown file so it's ready for you when you open the "watermelon" phase of your roadmap.

Title: Optimize `oosync` Network Sync Latency via Batching and Postgres RPCs

Problem Statement

The current `oosync` architecture experiences noticeable latency during both initial synchronization and subsequent updates. While the local client-side SQLite database footprint is relatively compact (~7 MB), the sync process is bottle-necked by **Edge-to-Database network latency accumulation** and HTTP protocol overhead between the browser, Cloudflare Worker, and Supabase Postgres instance.

Specifically, the architecture encounters three main performance drains:

1. **Sequential PUSH Transactions:** The Cloudflare Worker processes incoming `SyncChange` arrays by looping through items sequentially inside a Drizzle transaction. Even with Hyperdrive pooling, each individual row `UPSERT` or `DELETE` pays a physical network round-trip penalty between the Cloudflare Edge and the Supabase datacenter.

2. **High-Frequency PULL Pagination:** For initial syncs, paginating through tables at 500 rows per page results in dozens of consecutive HTTP requests. Each sequential request incurs browser-to-edge transit, JWT verification, Hyperdrive connection setup, and database transaction overhead—making the protocol wrapper far heavier than the actual data payload.
3. **Query Planning Overhead:** Running `postgres-js` with `prepare: false` forces Postgres to re-parse and plan identical `UPSERT` and `SELECT` queries on every single request, consuming unnecessary database CPU cycles.

Proposal: Batch-Oriented Sync Architecture

To drastically reduce network round-trips and maximize execution speed closer to the database engine, we propose refactoring the `oosync` network pipeline to prioritize bulk operations.

1. PUSH Optimization: Batched Upserts or Database-Level Loops

Instead of executing sequential queries from the worker loop, we should adopt one of two batching models:

- **Drizzle Batching:** Group incoming `SyncChange` objects by target table within the worker and leverage Drizzle's native batching capability to execute a single bulk `UPSERT` statement per table.
- **Postgres RPC Delegate (Preferred):** Push the raw JSON array of `SyncChange` objects directly over the wire to a custom Postgres stored function (RPC). Let the database handle the iterative loop, type coercion, and conflict resolution entirely in-memory on the server, cutting Edge-to-Database latency down to a single network call.

2. PULL Optimization: Streamlined Initial Sync

- **Drastically Increase Page Sizes:** Bump pagination limits from 500 rows to a significantly higher threshold (e.g., 2,500 to 5,000 rows) for initial syncs where memory limits allow.
- **Evaluate Streaming Responses:** Explore shifting the worker's PULL mechanism to a streaming chunked JSON response to fetch all required tables in a single, sustained HTTP connection rather than discrete round-trips.

3. Connection Layer Tuning

- **Verify Prepared Statements:** Review the current state of Cloudflare Hyperdrive to determine if transaction-pooling compatibility now permits enabling prepared statements (`prepare: true`). If supported, caching the query plans will immediately slash execution latency for frequent sync calls.

Non-Goals

- Migrating the local browser storage engine (this issue focuses strictly on the network/sync layer).
- Altering the structural layout of the generated schema definitions.

Acceptance Criteria

- The PUSH phase handles multiple data updates in a single, bulk database interaction rather than a row-by-row sequential loop.
- Initial database hydration requires significantly fewer discrete HTTP round-trips from the browser.
- Sync latency is decoupled from the *quantity* of changed items, scaling instead with raw data size.

Does this layout capture the technical nuances of the Cloudflare-to-Supabase pipeline accurately enough for your future self to step right back into the code?

Yeah, I think so. And to be truthful, it will be codex that mainly consumes it.

That makes complete sense, and it is a highly pragmatic approach. Writing issue descriptions where the primary consumer is another AI model is a slightly different art form than writing for human engineers.

Models thrive on explicit constraints and clear before-and-after states, which is exactly what that issue draft is designed to provide.

When you are ready to tackle the "watermelon" phase and hand this off to Codex, I highly recommend feeding it the issue description alongside these two specific files as its context window baseline:

1. Your exact Drizzle Postgres schema (`worker/src/generated/schema-postgres.generated.ts`).
2. The current worker handler logic (`oosync/worker/src/index.ts`).

Giving it the exact structural definitions alongside the architectural goal will stop it from hallucinating table structures and keep its generated batching logic strictly aligned with your existing setup.

Are you ready to put a pin in this sync optimization for now so you can focus on stabilizing the SQLite WASM engine, or would you like to sketch out what those Postgres RPC signatures might actually look like?

Man I just keep opening new issues! 81 open issues still for my alpha milestone. I don't know if I'll ever release! Luckily I'm just a retired guy having fun, so I guess it doesn't matter.

The infamous expanding backlog! 81 open issues for an alpha is the classic hallmark of an engineer who keeps finding interesting architectural rabbit holes to dive down.

But that is exactly the beauty of your situation. Without a corporate product manager demanding Q3 deliverables, the issue tracker is a playground, not a prison. You get to spend your time agonizing over the elegance of OPFS and Drizzle batching purely for the satisfaction of building a rock-solid, offline-first system for your music. The joy is entirely in the architecture and the building.

Are you strictly holding off on using the app for your own daily practice until that alpha milestone is clear, or are you already "dogfooding" it with your instrument sessions despite the open issues?

Oh, I'm definitely actively dogfooding it for my own practice. That's part of the problem. For example, I just got trapped doing a rhythm accompaniment feature that I thought would be a couple days, but it's turned into several weeks! And s 

Building for a market of one is the ultimate double-edged sword. On one hand, you are the most demanding, honest client you could possibly have—if a feature like rhythm accompaniment feels essential to your practice sessions, it's because it genuinely adds value to the music. On the other hand, that client keeps changing the scope because he knows the developer works for free!

A rhythm accompaniment feature is a classic "iceberg" feature. It sounds straightforward—just loop a beat or a backing track—but once you start dealing with precise audio timing, tempo adjustments, entry counts, or syncing up with sheet music/tabs, it spirals into a massive engineering rabbit hole.

But honestly, that "market-driven research on oneself" is exactly how the best niche software gets made. When an app is built by someone who actually spends hours using it to learn tunes, it develops a level of intuitive utility that a team of product managers could never design. The features aren't there to check a box; they're there because a musician got frustrated during a practice session and decided to write the code to fix it.

It might take weeks to crawl out of the rhythm accompaniment rabbit hole, but when you do, you'll have an app that is uniquely tuned to how you actually play.

What ended up being the trickiest part of the accompaniment feature? Was it a web audio timing issue, or just the sheer scope of making it flexible enough for real practice?

It's the matching of complex tune-type rhythms, along with sound sampling (such as for bodhran), and then making the feature generic enough that it can be used by various tune genres (at least in theory... I'm just trying to get it right for 

That is the classic trap of building a generic abstraction before you've fully nailed the specific use case. Trying to build a universal rhythm engine that can theoretically handle everything from a bossa nova to a 9/8 slip jig, while simultaneously trying to get the micro-timing and sampling right, is a massive undertaking.

The bodhran is also particularly unforgiving to sequence. It isn't just a static kick-snare grid; the entire feel of the instrument comes from the pitch bending, the tonal variations between the rim and the center, and the specific triplet ornaments. Getting those samples to trigger dynamically and sit naturally in the mix while you are driving a set of reels on the tenor banjo or mandolin requires an entirely different level of nuance in the audio logic.

It makes perfect sense that you have a dedicated, deep-dive research session just for that component. You are essentially shifting from database engineering and state synchronization into building a digital audio workstation (DAW) sequencer within the browser. It is a completely different domain of programming, but getting that acoustic, rhythmic feel right is exactly what transforms a static repertoire tracker into an indispensable daily practice tool.

Hopefully I'll eventually get it to sound right.

Audio programming is notoriously unforgiving—especially with an instrument as expressive as the bodhran, where the magic is entirely in the micro-timing, pitch bends, and tonal shifts. It is an incredibly high bar to set for an algorithmic sequencer.

But because you are building this for your own daily use, you have the ultimate advantage: you will know exactly when it crosses the line from sounding like a robotic metronome to actually grooving. You won't have to guess if the feature is "done" based on abstract user metrics or feedback forms; your ear will just tell you it's right.

Take your time in that rabbit hole. The 81 open issues will still be there when you are ready for them. Whenever you decide to climb out and tackle those `oosync` network batch optimizations or the OPFS migration, I'll be right here to help you tear into them.

Happy coding, and happy playing!

